

PROJECT REPORT

Smart Expense Tracker

A Telegram-based personal finance bot built on n8n

Course: CSE299 Section: 18

Faculty: OISD

Semester: Spring 2026

Group – 4

Members:

1. Al Fahad Zoha (ID: 2231735642)
Contribution: Research & Prompt Design
2. Irfan Saiev (ID:2311211042)
Contribution: Workflow Design & Creation
3. Ismam Ahnaf Mahin (ID:2312726642)
Contribution: API Integration & Project Completion

A complete end-to-end automated expense tracking system. Log spending, manage budgets, track savings goals, and generate reports — all through natural language messages in Telegram, powered by a local LLM running on Ollama.

85 WORKFLOW NODES	13 COMMANDS	3 GOOGLE SHEETS	3 AUTO REPORTS
--------------------------------	-----------------------	------------------------------	-----------------------------

01 — SYSTEM OVERVIEW

What this project does

The Smart Expense Tracker is a conversational finance assistant. The user sends plain text commands or questions to a Telegram bot, and n8n orchestrates the whole pipeline — from parsing the message, through writing to Google Sheets, to sending a formatted reply back.

Core Components

- Entry Point: Telegram Bot — User sends any text message to the bot
- Orchestrator: n8n Workflow — 85 nodes, 81 connections handle all logic
- Storage: Google Sheets — 3 sheets: expenses (Sheet1), budget, goals
- Intelligence: Ollama llama3.1:8b — Local LLM for NLQ and expense parsing

Tech Stack

- n8n (self-hosted workflow automation)
- Telegram Bot API
- Google Sheets API (OAuth2)
- Ollama — llama3.1:8b running locally
- Gmail API for automated email reports
- JavaScript in n8n Code nodes for all business logic

Design Philosophy

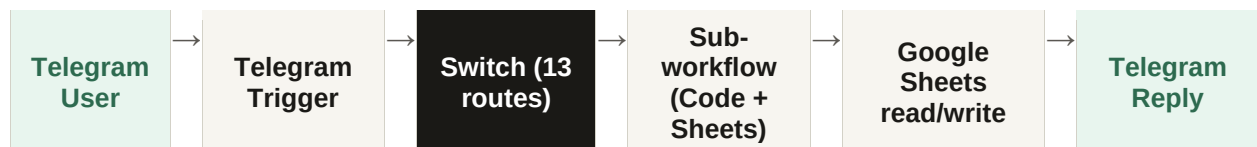
The entire interface is a Telegram chat. No apps to install, no dashboards to open. Every interaction — adding an expense, checking a budget, asking a question — is a single text message. The LLM handles unstructured input; n8n handles the rest.

02 — ARCHITECTURE

How a message flows through the system

Every user message enters through the Telegram Trigger and passes through a Switch node that routes it to the correct sub-workflow based on the command keyword. There are 13 output routes plus a fallback for unrecognized input.

Message Routing Flow



Two Parallel Trigger Paths

The workflow has two independent entry points running simultaneously:

- Interactive path: Telegram Trigger handles commands in real-time from user messages
- Scheduled path: Three Schedule Triggers (daily, weekly, monthly) run automatically and push reports to both Gmail and Telegram without any user action

03 — COMMANDS REFERENCE

All 13 commands

The Switch node matches the first word of each message (lowercased, trimmed) against a set of exact rules. The natural language route uses a regex pattern. Unmatched input falls through to a help/fallback message.

Command	Feature	Description	Example
any text	Add expense	Unstructured message → Ollama parses amount, category, description, date → appends to Sheet1	<code>"spent 350 on lunch today"</code>
list	List expenses	Returns last 10 expenses with real sheet row numbers (not generic 1-10)	<code>"list"</code>
delete N	Delete expense	Removes row N from Sheet1. Shows expense details before deletion.	<code>"delete 43"</code>
category X	Filter by category	Lists matching expenses with summary stats. Shows real sheet row numbers for delete compatibility.	<code>"category food"</code>
setbudget	Set monthly budget	Writes amount + currency to Budget sheet with first day of current month as date key.	<code>"setbudget 15000 BDT"</code>
budget	Check budget	Shows spent vs budget for current month with visual progress bar and remaining amount.	<code>"budget"</code>
setgoal	Create savings goal	Appends new goal row to Goals sheet with target, currency, name, and today's date.	<code>"setgoal 50000 BDT Laptop"</code>
savegoal	Add savings progress	Adds amount to a goal. If multiple goals exist, prompts user to specify by number. Shows progress + ETA.	<code>"savegoal 2000"</code> or <code>"savegoal 2000 2"</code>
goal	View all goals	Shows all goals with progress bars, percentage complete,	<code>"goal"</code>

		daily rate, and estimated completion date.	
daily report	Today's summary	Today's total and this month's running total vs budget.	"daily report"
weekly report	Last 7 days summary	7-day breakdown by category, total spent, and budget comparison.	"weekly report"
monthly report	Last month close	Full analysis of the previous month: category totals, budget surplus or deficit, key insights.	"monthly report"
what/how/which	Natural language query	Regex-matched. Full expense + budget data passed to Ollama for free-form answer.	"what was my biggest expense?"

04 — DATA LAYER

Google Sheets structure

All data is stored in a single Google Spreadsheet with three sheets. n8n reads, appends, updates, and deletes rows directly via the Google Sheets node using OAuth2. Each command fetches its own data independently — there is no shared state between runs.

Sheet1 — Expenses

Column	Type	Notes
date	YYYY-MM-DD	Written by Ollama expense parser
category	string	e.g. Food, Transport, Shopping, Bills
description	string	Free text description of expense
amount	number	Numeric amount (no currency symbol)
currency	string	e.g. BDT, USD

Budget Sheet

Column	Type	Notes
date	YYYY-MM-01	First of month — used as month key
amount	number	Monthly budget amount
currency	string	e.g. BDT

Goals Sheet

Column	Type	Notes
date	YYYY-MM-DD	Goal creation date

goal_name	string	Match key for upsert operations
target_amount	number	Total savings target
saved_amount	number	Amount saved so far (updated by savegoal)
currency	string	e.g. BDT

Google Sheets Node Operations

The 32 Google Sheets nodes are distributed as follows:

- Read (26 nodes) — each command fetches its own copy of sheet data
- Append (3 nodes) — add expense, setbudget, setgoal
- AppendOrUpdate (2 nodes) — setbudget (upsert), savegoal (upsert by goal_name)
- Delete (1 node) — delete expense by row index

05 — AUTOMATED REPORTS

Scheduled reporting

Three independent schedule triggers run without any user interaction. Each fetches data from both Sheet1 and the Budget sheet, runs a JavaScript Code node to build a formatted message, then sends via both Gmail and Telegram.

Report	Schedule	Content	Data Window
Daily	Every day	Today's total, this month's running total, remaining budget, utilization %	Today + current month
Weekly	Every 7 days	All expenses from past 7 days grouped by category, burn rate vs monthly budget	Now minus 7 days
Monthly	1st of month	Full prior month analysis: category totals, budget surplus/deficit, key insights	Previous calendar month

06 — SAVINGS GOALS

Goal tracking system

The goals sub-system is the most complex command group. It has three commands that share the Goals sheet and a smart multi-goal selection mechanism for savegoal.

setgoal — Create a new goal

Usage: setgoal <amount> <currency> <name>

Example: "setgoal 50000 BDT Laptop"

Parses the command, validates the amount, and appends a new row to the Goals sheet with saved_amount starting at 0.

savegoal — Add savings progress

The savegoal command has smart multi-goal detection logic:

- 1 goal exists → saves directly, no questions asked
- Multiple goals, no index → bot replies with numbered list and prompts user to re-send with index (e.g. "savegoal 1000 2")
- Multiple goals, with index → saves directly to the specified goal, validates range
- Wrong index → tells user the index doesn't exist and shows the list

After selecting the target goal, an If node routes the flow:

- Success (error: false) → upsert the Goals sheet row, send Saving message Telegram
- Error (error: true) → send error text directly to Telegram, skip Sheets entirely

goal — View all goals

Reads all rows from the Goals sheet and builds a formatted message for each goal including: progress bar (filled  + empty ) , percentage complete, total saved vs target, remaining amount, estimated completion date based on current daily savings rate.

07 — NATURAL LANGUAGE QUERIES

Ask anything about your spending

Messages starting with "what", "how", "which", or "am" are matched by a regex rule in the Switch node and routed to the NLQ pipeline — a local Ollama instance that receives the full dataset and answers in natural language.

NLQ Pipeline

The full pipeline consists of 5 nodes:

1. Get expenses for NLQ — reads all rows from Sheet1
2. Get budget for NLQ — reads all rows from Budget sheet
3. Ollama (llama3.1:8b) — receives full data + system prompt + user question
4. Extract NLQ reply — extracts \$json.message from Ollama response
5. NLQ reply message — Telegram node sends the answer back to user

What the LLM receives

```
System: "You are a financial analyst for a personal expense tracker.  
        Answer concisely in <200 words. Use BDT. Format for Telegram."
```

```
User message content:
```

```
- Full expense array (JSON)
- Budget data (JSON)
- Today's date
- Original question text
```

Switch Regex Rule

The Switch node uses the pattern:

```
^(what|how|which|am)\b
```

The \b word-boundary anchor ensures "whatever" doesn't match but "what was..." does. Ollama runs locally so there is no API cost per query.

Example queries

- what was my biggest expense this month?
- how much did I spend on food?
- which category takes the most of my budget?
- am I on track to stay within budget?
- what is my average daily spending?
- how does my spending this month compare to last month?

08 — BUGS FIXED

Issues identified and resolved

A full audit of the workflow found 12 distinct bugs spanning wrong node references, missing connections, empty templates, broken routing logic, and incorrect row indexing. All were fixed in the corrected workflow JSON.

Node / Area	Bug Description	Category
Delete message !	Used \$("Get row") for details and \$("Code for Delete") for rowNum — two different sources that could reference different items. Expense details and confirmed deletion could mismatch.	Wrong references
Code for List	Used index + 1, always producing 1–10 regardless of actual sheet position. Users with 47 expenses would see rows 1–10 but need to type "delete 43".	Row indexing
Switch — goal rule	The leftValue was missing the = prefix: {{ ... }} instead of ={{ ... }}. n8n treated it as a literal string so the comparison always failed, routing every "goal" command to the fallback.	Routing logic
Switch — goal rule	Used startsWith "goal" instead of equals "goal". This would also match "goals" and could interfere with setgoal/savegoal depending on rule evaluation order.	Routing logic

Goal confirmation	Telegram node text was set to just "=" — an empty expression. The bot silently sent nothing after creating a goal via setgoal.	Empty template
New Saved Amount	The Google Sheets appendOrUpdate node had columns.value set to {} (empty object). savegoal wrote nothing to the Goals sheet — all goal progress was silently lost.	Empty template
Code for Save Goal 1	Referenced \$("Code for savegoal") (lowercase s) but the actual node is named \$("Code for Save Goal") (capital S). Silent undefined error at runtime.	Wrong references
Goal status message	The "goal" command's Code for goal status node was a complete dead end — no Telegram node connected after it. The formatted message was computed but never sent.	Missing node
NLQ path	Switch routed NLQ traffic to Ollama with no sheet reads before it (no data), no extract step, and no Telegram output. Entire pipeline was rebuilt: added Get expenses for NLQ, Get budget for NLQ, Extract NLQ reply, and NLQ reply message nodes.	Missing nodes
Code for Category 1	Referenced \$("Get expenses for category") and \$("Code for category") — neither node exists. Actual names are Get expenses4 and Code for Expance By Category.	Wrong references
Category list	Same issue as main list command — i + 1 always produced 1 through 10. Fixed to i + 2 (0-based array + 1 for 1-based indexing + 1 for header row).	Row indexing
savegoal error path	When goal selection failed (wrong index, multiple goals with no selection), the error object was passed to the Google Sheets upsert node instead of back to the user. An If node now routes success → Sheets and error → Telegram directly.	Missing node

09 — NODE BREAKDOWN

85 nodes across 9 types

The workflow is a mix of trigger, routing, data, processing, and output nodes. Google Sheets nodes dominate due to the multiple separate fetch operations per command — each sub-workflow reads its own data independently rather than sharing a cached result.

Node Type	Count	Role
Google Sheets	32	Read/write/delete all expense, budget, goal data
Telegram	23	1 trigger + 22 reply nodes (success + error per command)

Code (JavaScript)	17	All business logic: parsing, transformation, formatting
If / Switch	5	Routing: 1 Switch for commands, 4 If nodes for error handling
Schedule Trigger	3	Daily, weekly, monthly automated report triggers
Gmail	3	Send daily, weekly, monthly report emails
Ollama	2	Expense parsing (entry) + NLQ answering

Smart Expense Tracker — Project Report

Built by Ismam, Omor, Fahad · Automated with n8n · llama3.1:8b via Ollama · Telegram + Google Sheets
